Assignment 4

Implementation of an Internet of Things node using the Adafruit Feather Huzzah32 Wi-Fi board and Micro Python

The goal was to turn an ESP32 Huzzah32 into a standalone embedded web server. No router, no external computer acting as a middleman, just the board creates its own WiFi access point, serves an HTML status page, and exposes a JSON API for reading sensor values and controlling output pins. Everything runs on MicroPython, directly on the chip.

Getting there was not straightforward. The board had to be flashed with MicroPython from scratch, the serial port fought back on macOS, the HTTP responses came out blank for reasons that took a while to find, the WiFi access point refused to broadcast until it did, and the RGB LED wiring went through more than one revision. This report covers all of it. What was built, how it works, and what it took to make it actually work.

The hardware on the breadboard: a push button, a red LED, an RGB LED, a potentiometer, and an MCP9808 temperature sensor. All wired to the ESP32, all readable or controllable through the API by the time this was done.

The report follows the four assignment tasks in order, then closes with a dedicated troubleshooting section covering the issues that came up and how they were resolved.

Web server code explanation

The server runs a raw HTTP/1.1 socket listener on port 80. When the board powers on, MicroPython automatically executes main.py. The first thing the file does is configure the WiFi access point, giving it the SSID and password defined at the top of the file. Once the AP is active, the code opens a TCP socket, binds it to 0.0.0.0 on port 80, and calls listen(5) to allow up to five queued connections before the loop starts.

```
10  ap = network.WLAN(network.AP_IF)
11  ap.active(True)
12  ap.config(essid=AP_SSID, authmode=3, password=AP_PASSWORD)
```

```
165  def run_server():
166      addr_info = socket.getaddrinfo("0.0.0.0", 80)[0][-1]
167      s = socket.socket()
168      s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
169      s.bind(addr_info)
170      s.listen(1)
171      print("listening on", addr_info)
```

The main loop calls socket.accept(), which blocks until a client connects. When a connection arrives, the server reads the incoming data in chunks and looks for the HTTP request line. That line contains the method (GET) and the path (for example /sensor/temp). The server extracts the path and passes it through a routing block built from if and elif statements. Each branch calls a specific handler function that builds and returns the response.

There is no web framework involved. No Flask, no Bottle, no Tornado. The routing is written by hand because on a microcontroller with 520 KB of RAM, importing a full framework is not realistic. Every byte of memory matters.

The response structure follows the HTTP specification directly. The server writes a status line first, then the headers, then a blank line, then the body. For routes that return HTML, the content type header is set to text/html. For API routes the content type is application/json. Every part of the response is encoded to bytes before being written to the socket. That encoding step caused the first major bug during development. The headers were being sent as plain Python strings rather than bytes, and the browser was receiving malformed data. The connection dropped silently with no error message. Once .encode("utf-8") was added to every write call, the page loaded correctly.

```
215             client_sock.send(header.encode("utf8"))
216             client_sock.send(html.encode("utf8"))
```

After the response is sent, the server closes the client socket and goes back to waiting. The server handles one request at a time. There is no threading and no async. On hardware this constrained, that is the correct decision. Attempting to handle concurrent connections on a device with this little memory would exhaust the heap quickly and crash the board.

The sensor reads happen inside the handler functions at request time. When a client asks for /sensor/temp, the handler reads the MCP9808 over I2C at that moment and formats the value as JSON before responding. The same pattern applies to the potentiometer, which is read from the ADC pin at request time, and to the button, which is read as a digital input. Nothing is cached and nothing runs in the background. The server is stateless between requests.

Sensor/Input display HTML

The HTML page at http://192.168.4.1/ is the human-facing side of the server. It loads in any browser on any device connected to the ESP32 access point and shows the current state of every sensor and input on the board without needing to refresh manually.

ESP32 IO status

Name	Value
btn1	0
rgb_g	0
rgb_b	1
led	1
rgb_r	0
pot	0
temp	26.69 C

The button state, read from GPIO 21, shows either pressed or not pressed depending on the current digital input level. The potentiometer value, read from GPIO 33 through the onboard ADC, shows a raw number between 0 and 4095 reflecting the current position of the dial. The temperature, read from the MCP9808 sensor over I2C, shows the current ambient temperature in degrees Celsius. The red LED state shows whether the output pin on GPIO 13 is currently high or low.

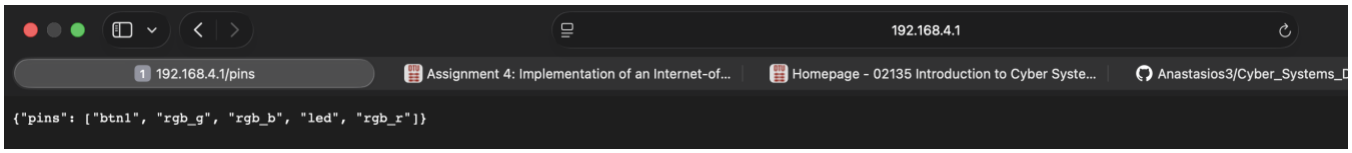
The HTML itself is generated as a Python string inside the handler function. There is no template engine and no external file. The sensor values are read at the moment the page is requested, formatted into the string, and sent as the response body. Every time the browser loads the page it gets a fresh reading.

The page also includes a basic HTML table to present the values clearly. Styling is kept minimal on purpose. The board has limited memory and sending a large HTML payload on every request wastes it. The table gives the reader a clean overview without adding unnecessary weight to the response.

One thing worth noting: the ADC on the ESP32 is not perfectly linear. The raw potentiometer reading is accurate enough for this assignment, but at the extreme ends of the dial the ADC compresses the range slightly. That is a hardware limitation of the chip, not something the code can fix.

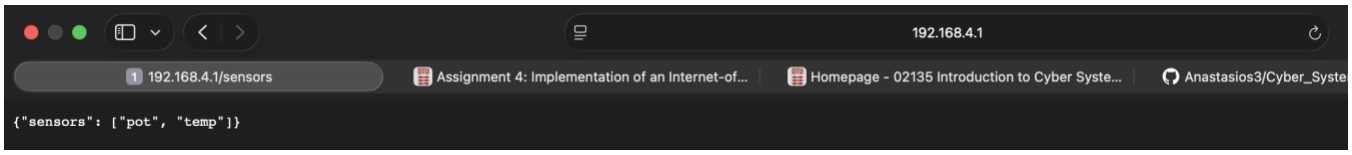
JSON Read API

The read API gives any HTTP client a structured way to query the board's current state. Instead of parsing HTML, a client sends a GET request to a specific path and gets back a JSON object. That makes the board programmable from other tools, not just a browser.



The available read endpoints are:

a. /pins returns the state of all digital pins at once. The response is a JSON object where each key is a pin name and each value is either 0 or 1.
b. /sensor/temp returns the current temperature reading from the MCP9808 in degrees Celsius, formatted as a JSON object with a single key.
c. /sensor/pot returns the raw ADC reading from the potentiometer on GPIO 33, a number between 0 and 4095.
d. /sensors returns all sensor values in one response, combining temperature and potentiometer into a single JSON object. This is useful when a client needs a full snapshot without making multiple requests.



Each endpoint follows the same pattern on the server side. The handler reads the relevant pin or sensor at request time, builds a Python dictionary, converts it with json.dumps(), and writes it to the socket with the correct application/json content type header. The client gets clean, parseable JSON every time.

```
123         value = func()
124         body = ujson.dumps({"name": name, "value": value})
125         return 200, "application/json", body
```

One design decision worth explaining: the endpoints are read-only and stateless. The board does not store previous readings and does not support filtering or pagination. Every request returns the current state and nothing else. For an embedded device this is the right tradeoff. Keeping state in memory on a microcontroller adds complexity and risk without much benefit for this use case.

```
request: GET /pin/led/set/low HTTP/1.1
client connected from ('192.168.4.2', 64915)
request: GET /pins HTTP/1.1
client connected from ('192.168.4.2', 53135)
request: GET /sensors HTTP/1.1
client connected from ('192.168.4.2', 53136)
client connected from ('192.168.4.2', 60993)
client connected from ('192.168.4.2', 60994)
request: GET /sensor/temp HTTP/1.1
```

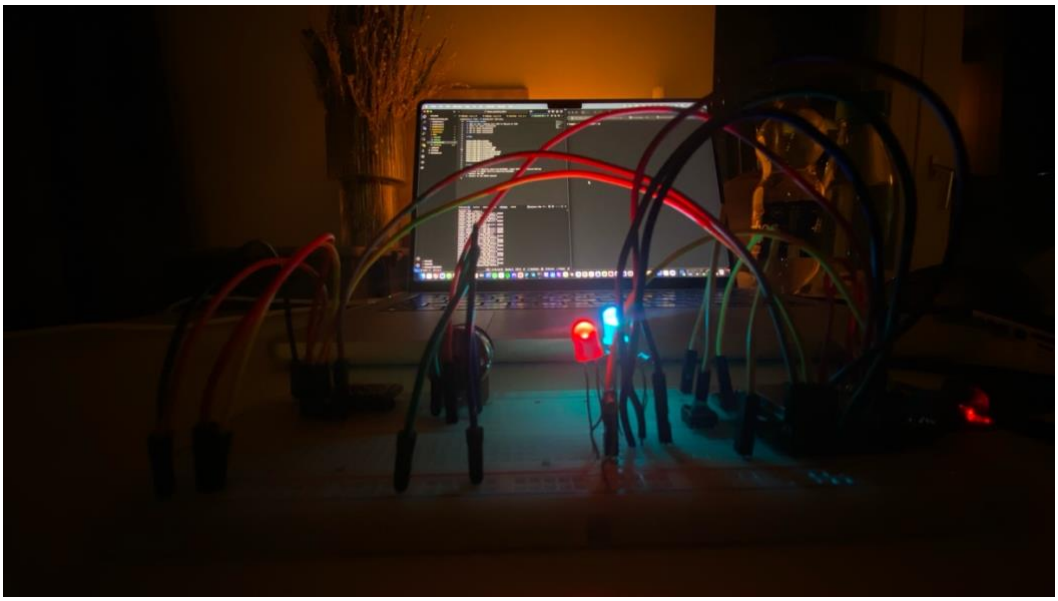
JSON Write API and RGB LED control

The write API extends the read API with endpoints that change the state of output pins rather than just reporting them. A GET request to the right path turns a pin on or off, sets the red LED, or drives the RGB LED to any color combination. The board stops being a passive sensor and becomes something you can actually control.

The red LED on GPIO 13 is controlled through two endpoints. Sending a request to /pin/led/set/high pulls the pin high and turns the LED on. Sending a request to /pin/led/set/low pulls it low and turns it off. The server reads the path, sets the pin accordingly using machine.Pin, and responds with a JSON confirmation containing the pin name and its new state.

```
client connected from ('192.168.4.2', 57039)
request: GET /pin/led/set/low HTTP/1.1
client connected from ('192.168.4.2', 61961)
request: GET /pin/led/set/low HTTP/1.1
client connected from ('192.168.4.2', 61962)
```

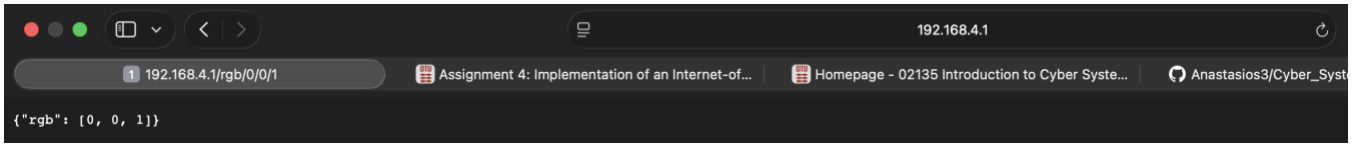
The RGB LED is controlled through the /rgb/R/G/B endpoint where R, G and B are each either 0 or 1. Sending /rgb/1/0/0 turns the red channel on and the others off. Sending /rgb/0/1/0 gives green. Sending /rgb/0/0/1 gives blue. The three values are extracted directly from the URL path, converted to integers, and written to the three GPIO pins assigned to the RGB LED channels.



```
153         rgb_r.value(1 if r > 0 else 0)
154         rgb_g.value(1 if g > 0 else 0)
155         rgb_b.value(1 if b > 0 else 0)
```

Getting the RGB LED to work took longer than expected. The initial wiring had the channels mapped to the wrong pins, so the colors came out wrong or nothing lit up at all. Once the pin assignments in the code matched the actual physical wiring on the breadboard, all three channels worked correctly. Red on GPIO 25, green on GPIO 26, blue on GPIO 27.

The write endpoints use the same socket handling pattern as everything else. The path is parsed, the action is taken, and a JSON response confirms what happened. There is no authentication and no rate limiting. On a private access point used for a course assignment that is acceptable. On anything exposed to a real network it would not be.



(additional) Troubleshooting – the odyssey

This section exists because the four chapters before this one make everything look cleaner than it was. What follows is what actually happened. Multiple days. Multiple coffee cups. One very patient AI assistant named Claude and Googling … a lot of googling.

Flashing MicroPython

The plan was simple: download firmware, flash it, done. The plan lasted about four minutes.

First problem was bootloader mode. The board ignores esptool completely if you do not hold BOOT and press RESET at exactly the right moment. Once that was figured out, the erase went fine. Then the wrong firmware was downloaded. The ESP32 Huzzah32 needs the plain generic binary, not the S2, not the S3, not anything with an

extra letter in the name. Once the right file was found, the flash wrote cleanly, hash verified, done. That was day one. Just the flashing.

Serial port conflicts

screen opened, printed nothing, and closed. Turns out macOS had already claimed the port silently in the background. lsof found the culprit, kill -9 removed it, and screen was quietly replaced with picocom which at least has the decency to say what it is doing. The >>> prompt appeared and felt like a much bigger win than it deserved to be.

The blank page

main.py uploaded, board reset, browser opened. White page. Not an error, just nothing. The response headers were being sent as plain Python strings instead of bytes. The browser received garbage, dropped the connection, and told nobody. One .encode("utf-8") call fixed it. That one took a day to find.

The access point that was always there

The WiFi network was not showing up in the Mac's scan. The code was read and reread. Claude read it too. Everything looked correct. Dropping into the REPL and running ap.active() showed it had been broadcasting the entire time. The Mac just needed a WiFi scan refresh. The network had been up for the better part of an afternoon while the code was being blamed for something it did not do.

RGB LED

The API worked. The HTML worked. The sensors worked. Then /rgb/1/0/0 was sent and the wrong color came on. The channels were wired to the wrong pins. Remapping the pin assignments in the code to match the actual physical wiring fixed it on the first try, which was equal parts satisfying and irritating.

By the end of it all, the board worked. Claude did not give up. The developer almost did, twice, but did not put that in the git commit messages.